



Batch 22 — Product Segmentation + Pricing Control Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:23

Batch 22 — Product Segmentation + Pricing Control Pack

Meat Lovers CIMS powered by YohPal

22A.

database/migrations/042_pricing_rules.sql

```
CREATE TABLE pricing_rules (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  product_category ENUM(
    'FOOD',
    'SOFT_DRINK',
    'ALCOHOLIC_DRINK'
  ) NOT NULL,

  minimum_margin_percent DECIMAL(5,2) DEFAULT 0,
  maximum_discount_percent DECIMAL(5,2) DEFAULT 0,

  rule_status ENUM(
    'ACTIVE',
    'INACTIVE'
  ) DEFAULT 'ACTIVE',

  notes TEXT,

  created_by BIGINT,

  FOREIGN KEY (created_by)
    REFERENCES users(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

22B.

database/migrations/043_price_change_audit.sql

```
CREATE TABLE price_change_audit (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  product_id BIGINT NOT NULL,

  old_selling_price DECIMAL(12,2),
  new_selling_price DECIMAL(12,2),

  old_cost_price DECIMAL(12,2),
  new_cost_price DECIMAL(12,2),

  changed_by BIGINT,
  change_reason TEXT,

  FOREIGN KEY (product_id)
    REFERENCES products(id),

  FOREIGN KEY (changed_by)
    REFERENCES users(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

22C.

database/migrations/044_margin_alerts.sql

```
CREATE TABLE margin_alerts (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  product_id BIGINT NOT NULL,

  product_category ENUM(
    'FOOD',
    'SOFT_DRINK',
    'ALCOHOLIC_DRINK'
  ) NOT NULL,

  selling_price DECIMAL(12,2),
  cost_price DECIMAL(12,2),
  margin_percent DECIMAL(5,2),

  alert_status ENUM(
    'OPEN',
    'REVIEWED',
    'RESOLVED',
    'CLOSED'
  ) DEFAULT 'OPEN',

  notes TEXT,

  FOREIGN KEY (product_id)
    REFERENCES products(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

22D.

database/seeds/017_pricing_rules_seed.sql

```
INSERT INTO pricing_rules (
  product_category,
  minimum_margin_percent,
  maximum_discount_percent,
  rule_status,
  notes,
  created_by
) VALUES
('FOOD', 35.00, 10.00, 'ACTIVE', 'Food should maintain at least 35% gross margin', 1),
('SOFT_DRINK', 25.00, 5.00, 'ACTIVE', 'Soft drinks should maintain at least 25% gross margin', 1),
('ALCOHOLIC_DRINK', 30.00, 5.00, 'ACTIVE', 'Alcoholic drinks should maintain at least 30% gross margin', 1);
```

22E. Update

backend/routes/api.php

Add routes:

```
$router->get('/api/pricing/rules', [ProductsController::class, 'pricingRules']);
$router->post('/api/pricing/rules', [ProductsController::class, 'createPricingRule']);

$router->get('/api/pricing/category-dashboard/{id}', [ProductsController::class, 'categoryDashboard']);

$router->post('/api/products/{id}/update-price', [ProductsController::class, 'updatePrice']);
$router->post('/api/products/{id}/deactivate', [ProductsController::class, 'deactivateProduct']);
$router->post('/api/products/{id}/activate', [ProductsController::class, 'activateProduct']);

$router->get('/api/products/price-change-audit', [ProductsController::class, 'priceChangeAudit']);
$router->get('/api/products/margin-alerts', [ProductsController::class, 'marginAlerts']);
$router->post('/api/products/margin-alerts/generate', [ProductsController::class, 'generateMarginAlerts']);
```

Note: the router currently supports {id} as numeric path segments. For category dashboards, use:

- 1 = FOOD
 - 2 = SOFT_DRINK
 - 3 = ALCOHOLIC_DRINK
-

22F. Update

backend/app/Controllers/ProductsController.php

Add imports:

```
use Support\Auth;
use Support\AuditLogger;
```

Add methods inside the class:

```
public function pricingRules(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        'SELECT
            pr.*,
            u.full_name AS created_by_name
        FROM pricing_rules pr
        LEFT JOIN users u ON u.id = pr.created_by
        ORDER BY pr.product_category ASC'
    );

    Response::success(['pricing_rules' => $stmt->fetchAll()]);
}

public function createPricingRule(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO pricing_rules
        (product_category, minimum_margin_percent, maximum_discount_percent, rule_status, notes, created_by)
        VALUES (:category, :minimum_margin, :maximum_discount, :status, :notes, :created_by)'
    );

    $stmt->execute([
        'category' => $data['product_category'],
        'minimum_margin' => $data['minimum_margin_percent'],
        'maximum_discount' => $data['maximum_discount_percent'],
        'status' => $data['rule_status'] ?? 'ACTIVE',
        'notes' => $data['notes'] ?? null,
        'created_by' => $user['id'],
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'PRICING',
        'CREATE_PRICING_RULE',
        null,
        null,
        $data
    );

    Response::success([], 'Pricing rule created');
}

public function categoryDashboard(int $id): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $category = match ($id) {
        1 => 'FOOD',
        2 => 'SOFT_DRINK',
        3 => 'ALCOHOLIC_DRINK',
        default => null,
    };

    if (!$category) {
        Response::error('Invalid category dashboard', 400);
    }

    $db = Database::connection();

    $products = $db->prepare(
        'SELECT
            id,
            product_name,
            product_category,
            selling_price,
            cost_price,
            is_active,
            CASE
                WHEN selling_price > 0 THEN ((selling_price - cost_price) / selling_price) * 100
            ELSE 0
            END AS margin_percent
        FROM products
        WHERE product_category = :category
        ORDER BY product_name ASC'
    );
}
```

```

$products->execute(['category' => $category]);

$summary = $db->prepare(
    'SELECT
        COUNT(*) AS product_count,
        SUM(CASE WHEN is_active = 1 THEN 1 ELSE 0 END) AS active_count,
        SUM(CASE WHEN is_active = 0 THEN 1 ELSE 0 END) AS inactive_count,
        AVG(CASE WHEN selling_price > 0 THEN ((selling_price - cost_price) / selling_price) * 100 ELSE 0
END) AS average_margin
    FROM products
    WHERE product_category = :category'
);

$summary->execute(['category' => $category]);

Response::success([
    'category' => $category,
    'summary' => $summary->fetch(),
    'products' => $products->fetchAll(),
]);
}

public function updatePrice(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
    $data = Request::body();
    $db = Database::connection();

    $oldStmt = $db->prepare('SELECT * FROM products WHERE id = :id LIMIT 1');
    $oldStmt->execute(['id' => $id]);
    $old = $oldStmt->fetch();

    if (!$old) {
        Response::error('Product not found', 404);
    }

    $newSelling = $data['selling_price'] ?? $old['selling_price'];
    $newCost = $data['cost_price'] ?? $old['cost_price'];

    $update = $db->prepare(
        'UPDATE products
        SET selling_price = :selling_price,
            cost_price = :cost_price
        WHERE id = :id'
    );

    $update->execute([
        'selling_price' => $newSelling,
        'cost_price' => $newCost,
        'id' => $id,
    ]);

    $audit = $db->prepare(
        'INSERT INTO price_change_audit
        (product_id, old_selling_price, new_selling_price, old_cost_price, new_cost_price, changed_by,
change_reason)
        VALUES (:product_id, :old_selling, :new_selling, :old_cost, :new_cost, :changed_by, :reason)'
    );

    $audit->execute([
        'product_id' => $id,
        'old_selling' => $old['selling_price'],
        'new_selling' => $newSelling,
        'old_cost' => $old['cost_price'],
        'new_cost' => $newCost,
        'changed_by' => $user['id'],
        'reason' => $data['change_reason'] ?? null,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'PRICING',
        'UPDATE_PRODUCT_PRICE',
        $id,
        $old,
        $data
    );

    Response::success([], 'Product price updated');
}

public function deactivateProduct(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->prepare(
        'UPDATE products SET is_active = 0 WHERE id = :id'
    );

```

```

);

$stmt->execute(['id' => $id]);

AuditLogger::log(
    (int) $user['id'],
    'PRODUCTS',
    'DEACTIVATE_PRODUCT',
    $id
);

Response::success([], 'Product deactivated');
}

public function activateProduct(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->prepare(
        'UPDATE products SET is_active = 1 WHERE id = :id'
    );

    $stmt->execute(['id' => $id]);

    AuditLogger::log(
        (int) $user['id'],
        'PRODUCTS',
        'ACTIVATE_PRODUCT',
        $id
    );

    Response::success([], 'Product activated');
}

public function priceChangeAudit(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        'SELECT
            pca.*,
            p.product_name,
            u.full_name AS changed_by_name
        FROM price_change_audit pca
        JOIN products p ON p.id = pca.product_id
        LEFT JOIN users u ON u.id = pca.changed_by
        ORDER BY pca.created_at DESC'
    );

    Response::success(['price_change_audit' => $stmt->fetchAll()]);
}

public function marginAlerts(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        'SELECT
            ma.*,
            p.product_name
        FROM margin_alerts ma
        JOIN products p ON p.id = ma.product_id
        ORDER BY ma.created_at DESC'
    );

    Response::success(['margin_alerts' => $stmt->fetchAll()]);
}

public function generateMarginAlerts(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
    $db = Database::connection();

    $stmt = $db->query(
        'SELECT
            p.id,
            p.product_category,
            p.selling_price,
            p.cost_price,
            pr.minimum_margin_percent,
            CASE
                WHEN p.selling_price > 0 THEN ((p.selling_price - p.cost_price) / p.selling_price) * 100
                ELSE 0
            END AS margin_percent
        FROM products p
        JOIN pricing_rules pr ON pr.product_category = p.product_category
        WHERE pr.rule_status = "ACTIVE"'
    );

```

```

        AND p.is_active = 1'
    );

    $products = $stmt->fetchAll();
    $created = 0;

    foreach ($products as $product) {
        if ((float) $product['margin_percent'] < (float) $product['minimum_margin_percent']) {
            $alert = $db->prepare(
                'INSERT INTO margin_alerts
                (product_id, product_category, selling_price, cost_price, margin_percent, notes)
                VALUES (:product_id, :category, :selling, :cost, :margin, :notes)'
            );

            $alert->execute([
                'product_id' => $product['id'],
                'category' => $product['product_category'],
                'selling' => $product['selling_price'],
                'cost' => $product['cost_price'],
                'margin' => $product['margin_percent'],
                'notes' => 'Margin is below configured minimum',
            ]);

            $created++;
        }
    }

    AuditLogger::log(
        (int) $user['id'],
        'PRICING',
        'GENERATE_MARGIN_ALERTS',
        null,
        null,
        ['alerts_created' => $created]
    );

    Response::success([
        'alerts_created' => $created,
    ], 'Margin alerts generated');
}

```

22G. Update

apps/admin-web/src/features/operations/api/operationsApi.js

Add methods:

```

pricingRules: async () => {
    const { data } = await apiClient.get('/pricing/rules')
    return data.data.pricing_rules
},

createPricingRule: async (payload) => {
    const { data } = await apiClient.post('/pricing/rules', payload)
    return data
},

categoryDashboard: async (id) => {
    const { data } = await apiClient.get(`/pricing/category-dashboard/${id}`)
    return data.data
},

updateProductPrice: async (id, payload) => {
    const { data } = await apiClient.post(`/products/${id}/update-price`, payload)
    return data
},

deactivateProduct: async (id) => {
    const { data } = await apiClient.post(`/products/${id}/deactivate`, {})
    return data
},

activateProduct: async (id) => {
    const { data } = await apiClient.post(`/products/${id}/activate`, {})
    return data
},

priceChangeAudit: async () => {
    const { data } = await apiClient.get('/products/price-change-audit')
    return data.data.price_change_audit
},

marginAlerts: async () => {
    const { data } = await apiClient.get('/products/margin-alerts')
}

```

```

    return data.data.margin_alerts
  },

  generateMarginAlerts: async () => {
    const { data } = await apiClient.post('/products/margin-alerts/generate', {})
    return data
  },

```

22H.

apps/admin-web/src/features/pricing/pages/PricingRulesPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function PricingRulesPage() {
  return (
    <OperationalFormPage
      title="Pricing Rules"
      subtitle="Set category-specific pricing and discount rules"
      queryKey="pricing-rules"
      queryFn={operationsApi.pricingRules}
      mutationFn={operationsApi.createPricingRule}
      initialForm={{
        product_category: 'FOOD',
        minimum_margin_percent: '',
        maximum_discount_percent: '',
        rule_status: 'ACTIVE',
        notes: '',
      }}
      buildPayload={({form}) => ({
        ...form,
        minimum_margin_percent: Number(form.minimum_margin_percent),
        maximum_discount_percent: Number(form.maximum_discount_percent),
      })
    })
    fields={[
      {
        name: 'product_category',
        label: 'Product Category',
        type: 'select',
        options: [
          { value: 'FOOD', label: 'Food' },
          { value: 'SOFT_DRINK', label: 'Soft Drink' },
          { value: 'ALCOHOLIC_DRINK', label: 'Alcoholic Drink' },
        ],
      },
      { name: 'minimum_margin_percent', label: 'Minimum Margin %', type: 'number' },
      { name: 'maximum_discount_percent', label: 'Maximum Discount %', type: 'number' },
      {
        name: 'rule_status',
        label: 'Status',
        type: 'select',
        options: [
          { value: 'ACTIVE', label: 'Active' },
          { value: 'INACTIVE', label: 'Inactive' },
        ],
      },
      { name: 'notes', label: 'Notes', type: 'textarea' },
    ]}
    columns={[
      { key: 'product_category', label: 'Category' },
      { key: 'minimum_margin_percent', label: 'Minimum Margin %' },
      { key: 'maximum_discount_percent', label: 'Max Discount %' },
      { key: 'rule_status', label: 'Status' },
      { key: 'created_by_name', label: 'Created By' },
    ]}
    submitLabel="Create Pricing Rule"
  )
}

```

22I.

apps/admin-web/src/features/pricing/pages/CategoryDashboardPage.jsx

```

import React, { useState } from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import DataTable from '../../../components/ui/DataTable'

```

```

import LoadingBlock from '../.../components/ui/LoadingBlock'
import StatusMessage from '../.../components/ui/StatusMessage'
import { operationsApi } from '../.../operations/api/operationsApi'

const categories = {
  food: { id: 1, label: 'Food' },
  soft: { id: 2, label: 'Soft Drinks' },
  alcohol: { id: 3, label: 'Alcoholic Drinks' },
}

export default function CategoryDashboardPage() {
  const [category, setCategory] = useState('food')

  const { data, isLoading, error } = useQuery({
    queryKey: ['category-dashboard', category],
    queryFn: () => operationsApi.categoryDashboard(categories[category].id),
  })

  return (
    <Page title="Product Category Dashboard" subtitle="Food, soft drinks, and alcoholic drinks dashboard">
      <div style={{ display: 'flex', gap: 10, flexWrap: 'wrap' }}>
        <Button onClick={() => setCategory('food')}>Food</Button>
        <Button onClick={() => setCategory('soft')}>Soft Drinks</Button>
        <Button onClick={() => setCategory('alcohol')}>Alcoholic Drinks</Button>
      </div>

      {isLoading ? <LoadingBlock label="Loading category dashboard..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {data ? (
        <>
          <div style={{ display: 'grid', gridTemplateColumns: 'repeat(4, minmax(0, 1fr))', gap: 16 }}>
            <Card title="Products"><strong>{data.summary.product_count}</strong></Card>
            <Card title="Active"><strong>{data.summary.active_count}</strong></Card>
            <Card title="Inactive"><strong>{data.summary.inactive_count}</strong></Card>
            <Card title="Avg Margin"><strong>{Number(data.summary.average_margin || 0).toFixed(2)}%</strong></Card>
          </div>

          <Card title={` ${categories[category].label} Products`}>
            <DataTable
              columns={[
                { key: 'product_name', label: 'Product' },
                { key: 'selling_price', label: 'Selling Price' },
                { key: 'cost_price', label: 'Cost Price' },
                { key: 'margin_percent', label: 'Margin %' },
                { key: 'is_active', label: 'Active' },
              ]}
              rows={data.products || []}
            />
          </Card>
        </>
      ) : null}
    </Page>
  )
}

```

22J.

apps/admin-web/src/features/pricing/pages/ProductPriceControlPage.jsx

```

import React, { useState } from 'react'
import { useMutation } from '@tanstack/react-query'
import Page from '../.../components/ui/Page'
import Card from '../.../components/ui/Card'
import Button from '../.../components/ui/Button'
import Input from '../.../components/ui/Input'
import StatusMessage from '../.../components/ui/StatusMessage'
import { operationsApi } from '../.../operations/api/operationsApi'

export default function ProductPriceControlPage() {
  const [form, setForm] = useState({
    product_id: '',
    selling_price: '',
    cost_price: '',
    change_reason: '',
  })

  const [success, setSuccess] = useState('')

  const mutation = useMutation({
    mutationFn: () =>
      operationsApi.updateProductPrice(form.product_id, {
        selling_price: Number(form.selling_price),

```



```

        cost_price: Number(form.cost_price),
        change_reason: form.change_reason,
    }},
    onSuccess: () => setSuccess('Price updated successfully'),
  })
}

return (
  <Page title="Product Price Control" subtitle="Update product price with audit trail">
    <Card title="Update Product Price">
      <StatusMessage success={success} error={mutation.error?.message} />

      <form
        onSubmit={(e) => {
          e.preventDefault()
          setSuccess('')
          mutation.mutate()
        }}
        style={{ display: 'grid', gap: 10, maxWidth: 560 }}
      >
        <Input placeholder="Product ID" value={form.product_id} onChange={(e) => setForm({ ...form,
product_id: e.target.value })} />
        <Input placeholder="New Selling Price" value={form.selling_price} onChange={(e) => setForm({ ...form,
selling_price: e.target.value })} />
        <Input placeholder="New Cost Price" value={form.cost_price} onChange={(e) => setForm({ ...form,
cost_price: e.target.value })} />
        <Input placeholder="Reason for change" value={form.change_reason} onChange={(e) => setForm({ ...form,
change_reason: e.target.value })} />
        <Button type="submit">Update Price</Button>
      </form>
    </Card>
  </Page>
)
}

```

22K.

apps/admin-web/src/features/pricing/pages/ProductActivationControlPage.jsx

```

import React, { useState } from 'react'
import { useMutation } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import Input from '../../../components/ui/Input'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function ProductActivationControlPage() {
  const [productId, setProductId] = useState('')
  const [success, setSuccess] = useState('')

  const deactivateMutation = useMutation({
    mutationFn: () => operationsApi.deactivateProduct(productId),
    onSuccess: () => setSuccess('Product deactivated'),
  })

  const activateMutation = useMutation({
    mutationFn: () => operationsApi.activateProduct(productId),
    onSuccess: () => setSuccess('Product activated'),
  })

  return (
    <Page title="Product Activation Control" subtitle="Deactivate or reactivate products">
      <Card title="Product Status Control">
        <StatusMessage
          success={success}
          error={deactivateMutation.error?.message || activateMutation.error?.message}
        />

        <div style={{ display: 'grid', gap: 10, maxWidth: 560 }}>
          <Input placeholder="Product ID" value={productId} onChange={(e) => setProductId(e.target.value)} />

          <div style={{ display: 'flex', gap: 10, flexWrap: 'wrap' }}>
            <Button onClick={() => activateMutation.mutate()}>Activate Product</Button>
            <button onClick={() => deactivateMutation.mutate()} style={dangerButton}>
              Deactivate Product
            </button>
          </div>
        </div>
      </Card>
    </Page>
  )
}

```

```

const dangerButton = {
  padding: '10px 14px',
  borderRadius: 10,
  border: '1px solid #bel23c',
  background: '#fff1f2',
  color: '#bel23c',
  cursor: 'pointer',
}

```

22L.

apps/admin-web/src/features/pricing/pages/PriceChangeAuditPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function PriceChangeAuditPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['price-change-audit'],
    queryFn: operationsApi.priceChangeAudit,
  })

  return (
    <Page title="Price Change Audit" subtitle="Full audit trail of product price changes">
      <Card>
        {isLoading ? <LoadingBlock label="Loading price audit..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'product_name', label: 'Product' },
              { key: 'old_selling_price', label: 'Old Selling' },
              { key: 'new_selling_price', label: 'New Selling' },
              { key: 'old_cost_price', label: 'Old Cost' },
              { key: 'new_cost_price', label: 'New Cost' },
              { key: 'changed_by_name', label: 'Changed By' },
              { key: 'change_reason', label: 'Reason' },
              { key: 'created_at', label: 'Date' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}

```

22M.

apps/admin-web/src/features/pricing/pages/MarginAlertsPage.jsx

```

import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function MarginAlertsPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['margin-alerts'],
    queryFn: operationsApi.marginAlerts,
  })

  const mutation = useMutation({
    mutationFn: operationsApi.generateMarginAlerts,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['margin-alerts'] }),
  })

  return (

```

```

<Page title="Margin Alerts" subtitle="Products below category minimum margin">
  {mutation.error ? <StatusMessage error={mutation.error.message} /> : null}

  <Card title="Generate Alerts">
    <Button onClick={() => mutation.mutate()}>Generate Margin Alerts</Button>
  </Card>

  <Card title="Alerts">
    {isLoading ? <LoadingBlock label="Loading margin alerts..." /> : null}
    {error ? <StatusMessage error={error.message} /> : null}

    {!isLoading && !error ? (
      <DataTable
        columns={[
          { key: 'product_name', label: 'Product' },
          { key: 'product_category', label: 'Category' },
          { key: 'selling_price', label: 'Selling Price' },
          { key: 'cost_price', label: 'Cost Price' },
          { key: 'margin_percent', label: 'Margin %' },
          { key: 'alert_status', label: 'Status' },
          { key: 'notes', label: 'Notes' },
        ]}
        rows={data || []}
      />
    ) : null}
  </Card>
</Page>
)
}

```

22N. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```

import PricingRulesPage from '../features/pricing/pages/PricingRulesPage'
import CategoryDashboardPage from '../features/pricing/pages/CategoryDashboardPage'
import ProductPriceControlPage from '../features/pricing/pages/ProductPriceControlPage'
import ProductActivationControlPage from '../features/pricing/pages/ProductActivationControlPage'
import PriceChangeAuditPage from '../features/pricing/pages/PriceChangeAuditPage'
import MarginAlertsPage from '../features/pricing/pages/MarginAlertsPage'

```

Add routes:

```

<Route path="/pricing/rules" element={<PricingRulesPage />} />
<Route path="/pricing/category-dashboard" element={<CategoryDashboardPage />} />
<Route path="/pricing/price-control" element={<ProductPriceControlPage />} />
<Route path="/pricing/product-activation" element={<ProductActivationControlPage />} />
<Route path="/pricing/price-audit" element={<PriceChangeAuditPage />} />
<Route path="/pricing/margin-alerts" element={<MarginAlertsPage />} />

```

22O. Update

apps/admin-web/src/components/common/SidebarNav.jsx

Add links:

```

['Pricing Rules', '/pricing/rules'],
['Category Dashboard', '/pricing/category-dashboard'],
['Price Control', '/pricing/price-control'],
['Product Activation', '/pricing/product-activation'],
['Price Audit', '/pricing/price-audit'],
['Margin Alerts', '/pricing/margin-alerts'],

```

22P.

docs/PRODUCT_SEGMENTATION_PRICING_CONTROL_PACK.md

Batch 22 – Product Segmentation + Pricing Control Pack

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch strengthens product segmentation, pricing control, margin protection, and product activation governance.

Product Categories

The system enforces three commercial categories:

1. Food
2. Soft drinks
3. Alcoholic drinks

Pricing Rules

Pricing rules define:

- minimum margin percentage
- maximum discount percentage
- rule status
- notes

Category Dashboards

Management can view dashboards for:

- food
- soft drinks
- alcoholic drinks

Each dashboard shows:

- product count
- active count
- inactive count
- average margin
- product-level margin

Price Change Audit

Every price update records:

- old selling price
- new selling price
- old cost price
- new cost price
- user who changed it
- reason for change

Margin Alerts

The system can generate margin alerts when:

```
```text
product margin < category minimum margin
```

# Inactive Product Control

Products can be:

- activated
- deactivated

This helps prevent unavailable products from appearing in POS and reports.

# Business Value

This batch helps Meat Lovers:

- protect gross margins
- separate food, soft drinks, and alcohol performance
- prevent unauthorized price changes
- detect loss-making items
- control which products are available for sale

---

## # Batch 22 Outcome

Product segmentation + pricing control now includes:

- ✅ category-specific pricing rules
- ✅ food dashboard
- ✅ soft drinks dashboard
- ✅ alcoholic drinks dashboard
- ✅ margin alerts

✅ inactive product control

✅ price-change audit trail

# Next Smart Move

Build **Batch 23 – CRM + Customer Retention Pack**, including customer segmentation, visit history, loyalty points, feedback follow-up, birthday/anniversary reminder placeholder, abandoned website lead follow-up, and repeat-customer dashboard.